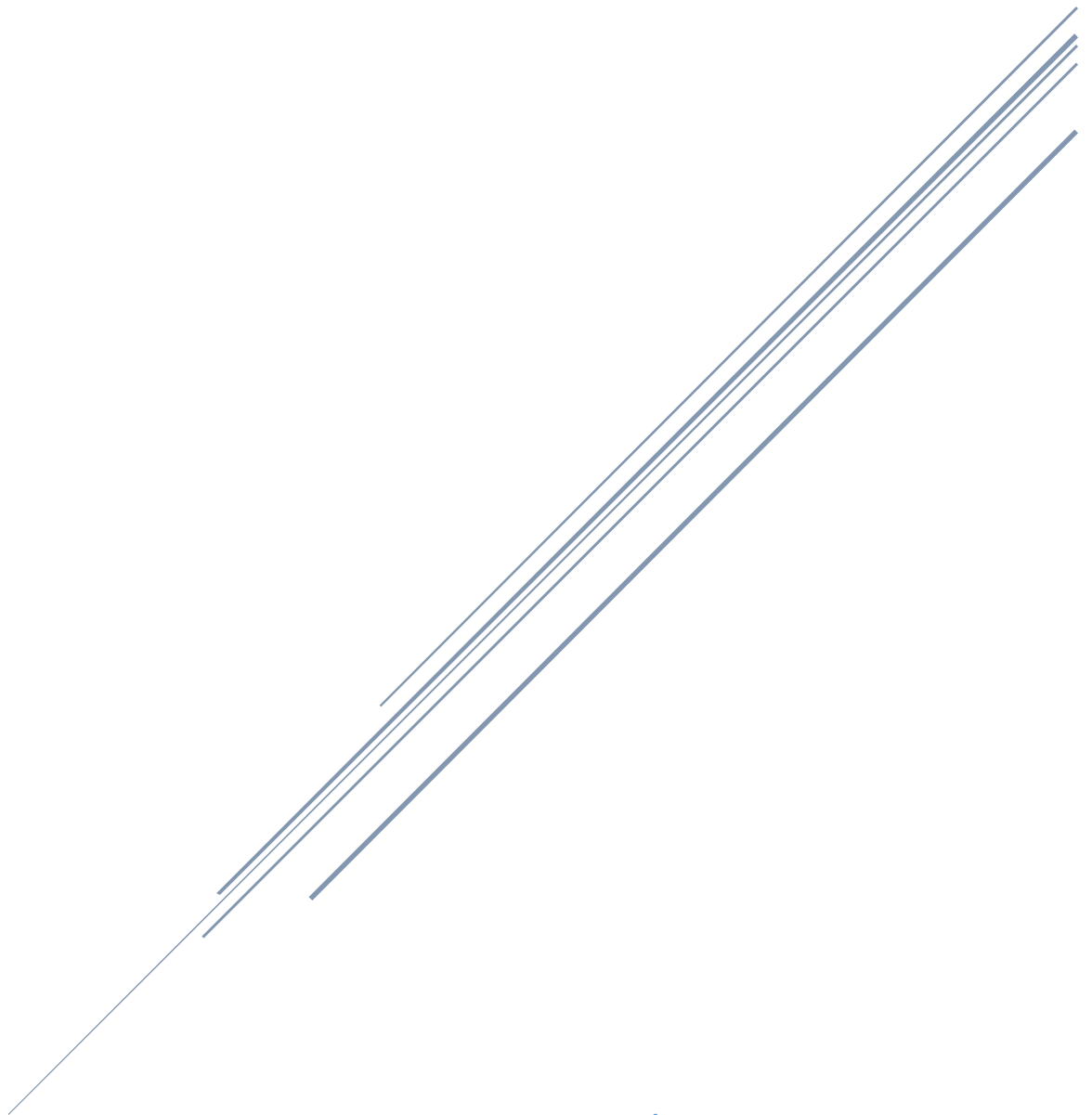


CODE AND SUMMARY OF CLASS 11

Sahir Ahmed Sheikh

Saturday (2 – 5)



Teachers:

Muhammad Bilal And Ali Aftab Sheikh

Code And Summary Of Class 11 – Saturday (2 – 5) | Quarter 3

Introduction

This document provides a detailed summary of Class 11 of Quarter 3, held on Saturday from 2 to 5 PM, led by Sir Ali Aftab Sheikh and Sir Muhammad Bilal. As the final class of Quarter 3, the session concluded the Object-Oriented Programming (OOP) curriculum, covering remaining topics from notebooks 12 and 13, including **Method Resolution Order (MRO)**, **decorators**, **callable objects**, **meta classes**, and **the singleton design pattern**. The class combined theoretical discussions, coding demonstrations in Python, and motivational guidance to prepare students for the upcoming challenges of Quarter 4.

The class addressed the following key objectives:

- Concluding Quarter 3 by covering remaining OOP topics, ensuring readiness for the viva.
- Introducing advanced Python concepts like decorators, callable objects, meta classes, and singleton patterns.
- Preparing students for Quarter 4, which will focus on AI agents and require independent learning.
- Emphasizing resilience, self-learning, and practical coding for industry relevance.

Students were informed that Quarter 3 officially ended with this class. Quarter 4, starting next week, was highlighted as more demanding, with a focus on AI agents and cloud computing, requiring students to adopt a self-directed learning approach.

Topics Covered

1. Class Updates and Quarter 3 Conclusion

- **Quarter 3 Wrap-Up:** Class 11 marked the end of Quarter 3, with a focus on revising notebooks 12 and 13. Students were praised for their efforts but warned that Quarter 4 will introduce advanced topics like AI agents, requiring immediate preparation.
- **Quarter 4 Logistics:** Due to heat concerns in June and July, class timings were adjusted to 3:30–6:30 PM on Saturdays, avoiding the hotter 2–5 PM slot. Night classes (7–10 PM) were considered but deemed too late.
- **Motivational Advice:** Sir Ali emphasized resilience, sharing the importance of consistent effort over immediate results. Students were encouraged to practice daily, explore repositories like the Open AI Agent SDK, and prepare for industry-relevant skills.

*2. Revision of OOP Concepts**:*

The session reviewed key OOP concepts from previous classes, reinforcing class and instance variables, composition, and aggregation, before diving into new topics. The class object relationship was reiterated using the blueprint analogy:

- **Class:** A template defining attributes and methods, e.g., a bank's account structure.
- **Object:** A specific instance of a class, e.g., an individual bank account.

3. Advanced Python Concepts

3.1 Method Resolution order (MRO)

- **Definition:** MRO defines the order in which Python searches for methods in a class hierarchy, using the C3 linearization algorithm.
- **Real-World Analogy:** Tracing a family tree from child to ancestors to determine inheritance.

- **Benefits:** Ensures correct method lookup in inheritance, especially in multiple inheritance scenarios.
- **Coding Example:**

```
class Car:
    pass

print(Car.mro()) # Output: [<class '__main__.Car'>, <class 'object'>]
```

- **Explanation:** The `mro()` method returns a list starting with the current class (`Car`) and ending with the base class (`object`), showing the method lookup order.

3.2 Decorators

- **Definition:** Decorators are functions that modify or extend the behavior of other functions or methods without altering their code.
- **Real-World Analogy:** Adding a decorative frame around a painting to enhance its appearance.
- **Types:** Function decorators, class decorators, and property decorators (e.g., `@property`).
- **Coding Example (Function Decorator):**

```
def star_decorator(func):
    def wrapper():
        print("*" * 5)
        func()
        print("*" * 5)
    return wrapper

@star_decorator
def say_hello():
    print("hello")

say_hello()
# Output:
# *****
# hello
# *****
```

- **Explanation:** `star_decorator` modifies the behavior of `say_hello()` by wrapping it with a wrapper function that adds pre- and post-processing (printing stars). This demonstrates **function wrapping and behavior extension** using Python decorators. When `say_hello()` is called, the decorator ensures stars are printed before and after the original output.
- **Coding Example (Class Decorator):**

```
class ObjectCounter:
    def __init__(self, cls):
        self.cls = cls
        self.count = 0
    def __call__(self, *args, **kwargs):
        self.count += 1
        print(f"{self.cls.__name__}_object_created_{self.count}_time(s)")
        return self.cls(*args, **kwargs)

@ObjectCounter
class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.balance = balance

account1 = BankAccount("Arf", 5000) # Output: BankAccount object
                                     created 1 time(s)
account2 = BankAccount("Ali", 3000) # Output: BankAccount object
                                     created 2 time(s)
```

- **Explanation:** The `ObjectCounter` class decorator tracks how many times `BankAccount` instances are created, showcasing class-based decoration.
- **Coding Example (Property Decorator):**

```

class Person:
    def __init__(self, name):
        self._name = name
    @property
    def name(self):
        return self._name
    @name.setter
    def name(self, new_name):
        self._name = new_name
    @name.deleter
    def name(self):
        print("Deleting name")
        del self._name

p = Person("Bob")
print(p.name) # Output: Bob
p.name = "Charlie" # Update via setter
print(p.name) # Output: Charlie
del p.name # Output: Deleting name

```

- **Explanation:** The @property decorator enables getter, setter, and deleter methods, allowing controlled access to the private _name attribute.

3.3 Callable Objects

- **Definition:** An object is callable if it implements the __call__ method, allowing it to be invoked like a function.
- **Real-World Analogy:** A vending machine that dispenses items when a button (method) is pressed.
- **Coding Example:**

```

class MyClass:
    def __call__(self):
        print("hello")

obj = MyClass()
print(callable(obj)) # Output: True
obj() # Output: hello

```

- **Explanation:** The `__call__` method makes obj callable, printing “hello” when invoked. The `callable()` function confirms this.

3.4 Meta Classes

- **Definition:** A meta class is a class of a class, defining how a class behaves. The default meta class in Python is type.
- **Real-World Analogy:** A factory’s blueprint (meta class) dictates how machines (classes) are built.
- **Coding Example:**

```
class Meta(type):
    def __new__(cls, name, bases, dct):
        print(f"Creating class {name}")
        return super().__new__(cls, name, bases, dct)

class MyClass(metaclass=Meta):
    pass

# Output: Creating class MyClass
```

- **Explanation:** The Meta class customizes class creation, printing a message when MyClass is defined, using the `__new__` method.

3.5 Singleton Design Pattern

- **Definition:** The singleton pattern ensures a class has only one instance, reusing it for all requests.
- **Real-World Analogy:** A single government office issuing passports, serving all citizens.
- **Coding Example:**

```
class Singleton:
    _instance = None
    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

s1 = Singleton()
s2 = Singleton()
print(s1 == s2) # Output: True
```

- **Explanation:** The `__new__` method ensures only one Singleton instance is created, with `s1` and `s2` referencing the same object.

4. Practical Coding and Debugging

- **Google Colab:** Used for coding exercises on decorators, callable objects, and singletons, with students encouraged to practice notebook 13 examples.
- **Debugging Example:** Addressed issues like incorrect argument handling in decorators (e.g., `*args`, `**kwargs`) and missing `__call__` methods, reinforcing robust coding practices.

5. Key Insights and Takeaways

- **OOP Mastery:** Advanced concepts like MRO, decorators, and meta classes enable scalable, modular code, preparing students for Quarter 4's AI projects.
 - **Self-Learning:** Quarter 4 will reduce spoon-feeding, requiring students to follow provided repositories (e.g., Open AI Agent SD) and learn independently.
 - **Industry Relevance:** Quarter 4's focus on AI agents aligns with high-demand skills, offering significant career opportunities.
 - **Resilience:** Sir Ali emphasized overcoming challenges like heat and scheduling, urging students to maintain effort and build portfolios.
-

Important Reminder

- The next class (May 31, 2025) marks the start of Quarter 4, focusing on AI agents. Attendance is mandatory, and incomplete assignments may affect Quarter 4 roles.
- Stay consistent, ask questions, and practice daily to master Python and succeed in Quarter 4.

Final Words

Your responsibility is to learn actively, participate confidently, and practice consistently. Quarter 4 will demand greater effort with advanced topics like AI and cloud computing. Embrace challenges, complete assignments, and build your portfolio to seize IT industry opportunities. Stay focused, and may Allah guide you.

Allah Hafiz – See you in Quarter 4!

Repo Link: [GIAIC-Sat-Afternoon](#)

This repository contains all class codes, assignments, and summaries of Quarter 3. Review each class's code and summary to prepare for Quarter 4

Thank You for Reading!

Hope you understood Class 11 well.

"Education is not the filling of a pail, but the lighting of a fire." – *William Butler Yeats*